



Nombre: **Lezly Martinez**

Fecha: **28 de septiembre de 2026**

Curso: **Física computacional I**

Docente: **Nestor Arias**

1. Actividad: equilibrio y puntos prometidos en un sistema de cargas eléctricas no lineal

1.1. Contexto físico

Se tienen dos cargas fijas $Q_1 = +4 \mu\text{C}$ en el origen $(0, 0)$ y $Q_2 = +1 \mu\text{C}$ en el punto $(2, 0)$ m. Una tercera carga móvil $q = +1 \mu\text{C}$ se mueve bajo la acción de ambas y de un potencial eléctrico externo no lineal

$$V_{ext}(x, y) = A(x^4 + y^4), \quad A = 0,5 \text{ V/m}^4. \quad (1)$$

La fuerza neta sobre q es

$$\vec{F}_{net}(x, y) = \vec{F}_{cargas}(x, y) - q \nabla V_{ext}(x, y). \quad (2)$$

Con $k = 8,99 \times 10^9 \text{ N m}^2/\text{C}^2$, $r_1 = \sqrt{x^2 + y^2}$ y $r_2 = \sqrt{(x-2)^2 + y^2}$, la fuerza de Coulomb (repulsiva, pues todas las cargas son positivas) es

$$\vec{F}_{cargas} = kq \left[Q_1 \frac{(x, y)}{r_1^3} + Q_2 \frac{(x-2, y)}{r_2^3} \right], \quad \nabla V_{ext} = 4A(x^3, y^3). \quad (3)$$

2. Parte A: método de relajación multivariable (2 variables)

2.1. Sistema de ecuaciones no lineales

Imponiendo $\vec{F}_{net} = 0$ en (2) y dividiendo por q (que aparece como factor común y no interviene en el equilibrio) se obtiene el sistema $f_1(x, y) = 0$, $f_2(x, y) = 0$:

$$f_1(x, y) = k \left[\frac{Q_1 x}{r_1^3} + \frac{Q_2 (x-2)}{r_2^3} \right] - 4A x^3 = 0, \quad (4)$$

$$f_2(x, y) = k \left[\frac{Q_1 y}{r_1^3} + \frac{Q_2 y}{r_2^3} \right] - 4A y^3 = 0. \quad (5)$$

Es un sistema de dos ecuaciones no lineales interdependientes, porque r_1 y r_2 dependen de ambas variables.

2.2. Reorganización en la forma $x = g_1(x, y)$, $y = g_2(x, y)$

Formulación 1 (raíz cúbica). El término no lineal es un cubo puro ($4Ax^3$ y $4Ay^3$), de modo que se puede despejar cada variable directamente pasando ese término al lado izquierdo:

$$x = g_1(x, y) = \sqrt[3]{\frac{k}{4A} \left[\frac{Q_1 x}{r_1^3} + \frac{Q_2 (x-2)}{r_2^3} \right]}, \quad y = g_2(x, y) = \sqrt[3]{\frac{k}{4A} \left[\frac{Q_1 y}{r_1^3} + \frac{Q_2 y}{r_2^3} \right]}. \quad (6)$$

Aquí $\sqrt[3]{\cdot}$ es la raíz cúbica real (con signo).

Formulación 2 (lineal). Sumando y restando la propia variable en cada ecuación se obtiene otra reorganización válida, pues en el punto fijo $f_1 = f_2 = 0$:

$$x = g_1(x, y) = x + \alpha f_1(x, y), \quad y = g_2(x, y) = y - \alpha f_2(x, y), \quad \alpha > 0. \quad (7)$$

El signo de cada componente se elige para que la derivada de g respecto a su propia variable tenga módulo menor que 1 cerca del equilibrio (se justifica en la sección de estabilidad).

2.3. Método de relajación en dos variables

Con un factor de relajación ω se itera

$$x^{(n+1)} = (1 - \omega) x^{(n)} + \omega g_1(x^{(n)}, y^{(n)}), \quad (8)$$

$$y^{(n+1)} = (1 - \omega) y^{(n)} + \omega g_2(x^{(n)}, y^{(n)}), \quad (9)$$

partiendo de $(x_0, y_0) = (1, 0, 0, 5)$ y deteniéndose cuando

$$\max(|x^{(n+1)} - x^{(n)}|, |y^{(n+1)} - y^{(n)}|) < \epsilon = 10^{-6}. \quad (10)$$

Con $\omega = 1$ es la iteración de punto fijo simple; con $\omega \neq 1$ es la relajación (o sub-relajación si $\omega < 1$).

2.4. Resultados

Formulación	Parámetros	Iteraciones	Punto final (x, y)
Raíz cúbica (6)	$\omega = 1, 0$	46	(5,9465, 6,2531)
Raíz cúbica (6)	$\omega = 0, 5$	33	(5,9465, 6,2531)
Raíz cúbica (6)	$\omega = 0, 2$	81	(5,9465, 6,2531)
Lineal (7)	$\alpha = 10^{-5}, \omega = 1$	23	(1,3333, 0)
Lineal (7)	$\alpha = 3 \times 10^{-5}, \omega = 1$	1765	(7,65, 0)

Tabla 1: Iteraciones necesarias desde $(1, 0, 0, 5)$ con $\epsilon = 10^{-6}$.

El sistema (4)–(5) tiene varios equilibrios. Se clasifican con los valores propios del hessiano de la energía potencial $U(x, y) = q \left[\frac{kQ_1}{r_1} + \frac{kQ_2}{r_2} + A(x^4 + y^4) \right]$ (ambos positivos: mínimo estable; de signos opuestos: punto silla):

Equilibrio (x^*, y^*)	Tipo
(1,3333, 0)	punto silla (estable en x , inestable en y)
(0,3802, $\pm 7,3939$)	punto silla
(5,9465, $\pm 6,2531$)	mínimo estable

Tabla 2: Equilibrios del sistema y su clasificación.

La trayectoria de convergencia en el plano (x, y) para dos de las formulaciones se muestra en la Figura 1, y el mapa de energía potencial con los equilibrios superpuestos en la Figura 2.

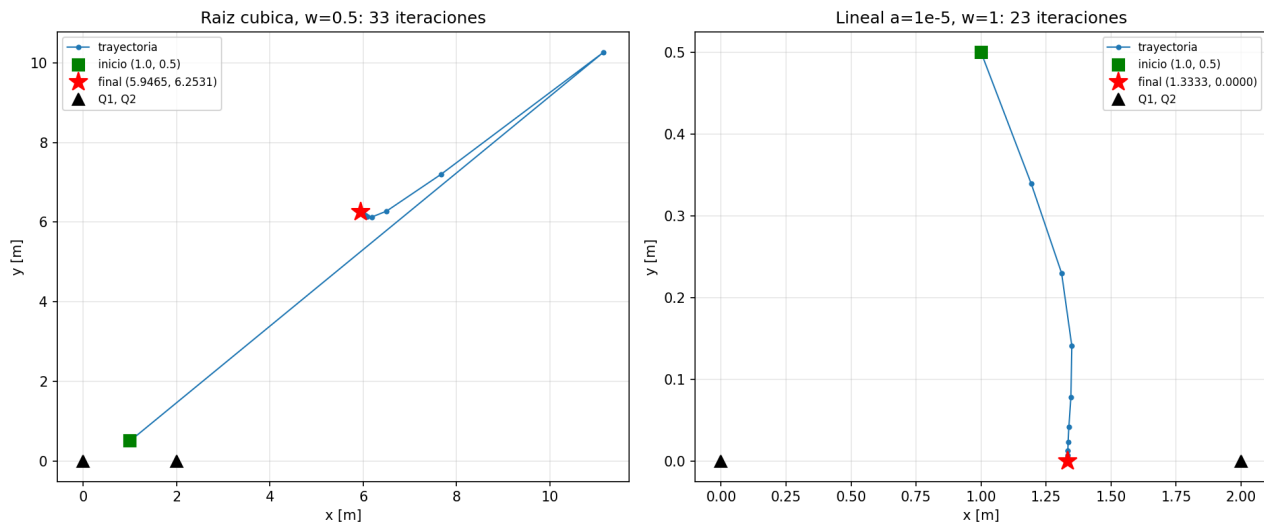


Figura 1: Trayectoria de convergencia en el plano (x, y) desde $(1, 0, 0.5)$: raíz cúbica con $\omega = 0.5$ (izquierda) y formulación lineal con $\alpha = 10^{-5}$ (derecha).

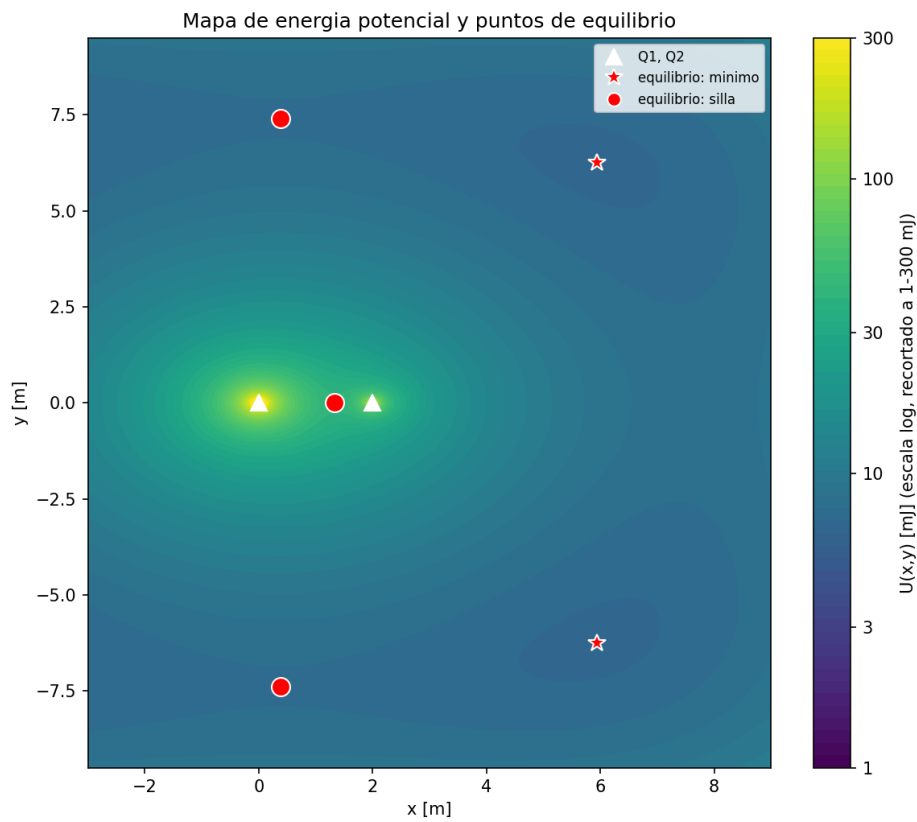


Figura 2: Mapa de energía potencial $U(x, y)$ (escala logarítmica, recortada a 1–300 mJ) con los puntos de equilibrio. Las estrellas son mínimos y los círculos, puntos silla; los triángulos son Q_1 y Q_2 .

3. Parte B: búsqueda binaria (bisección) en 1D

3.1. Función de energía

Con q restringida al eje x ($y = 0$, $0 < x < 2$), la energía potencial total es

$$U(x) = q \left[\frac{kQ_1}{x} + \frac{kQ_2}{2-x} + Ax^4 \right], \quad g(x) = U(x) - U_0. \quad (11)$$

Se busca la raíz de $g(x)$ en $[0,1, 1,9]$ con precisión $\Delta x < 10^{-5}$.

3.2. El valor $U_0 = 15$ mJ no es alcanzable

Evaluando $U(x)$ en el intervalo se obtiene

$$U_{\min} \approx 40,46 \text{ mJ } (x \approx 1,3), \quad U(0,1) = 364,33 \text{ mJ}, \quad U(1,9) = 108,83 \text{ mJ}. \quad (12)$$

Como $U(x) \geq 40,46$ mJ en todo el intervalo, $g(x) = U(x) - 15$ mJ es positiva en todo $[0,1, 1,9]$: no hay cambio de signo y el método de bisección no tiene raíz que encontrar (el programa lo detecta y lo reporta). Interpretar las cargas en nC tampoco lo resuelve, porque $U \propto Qq$ y entonces $U \leq 3,6 \times 10^{-4}$ mJ, que queda muy por debajo de 15 mJ.

3.3. Elección de un U_0 que sí funciona

Para que $g(0,1)$ y $g(1,9)$ tengan signos opuestos debe cumplirse $108,83 \text{ mJ} < U_0 < 364,33 \text{ mJ}$. Se toma

$$U_0 = 150 \text{ mJ}, \quad (13)$$

que cumple la condición y da una raíz única en la rama izquierda (en la rama derecha U no supera 108.83 mJ).

3.4. Resultados

La bisección con $[a, b] = [0,1, 1,9]$ converge a

$$x^* = 0,248223 \text{ m}, \quad U(x^*) = 150,0018 \text{ mJ}, \quad (14)$$

en **18 iteraciones**. Esto coincide con la cota teórica, ya que el ancho del intervalo se reduce a la mitad en cada paso:

$$n \geq \log_2 \frac{b-a}{\Delta x} = \log_2 \frac{1,8}{10^{-5}} \approx 17,5 \Rightarrow n = 18. \quad (15)$$

3.5. Comparación con la Parte A

Método	Iteraciones	Garantía de convergencia
Bisección (Parte B, $\Delta x < 10^{-5}$)	18	Sí, si hay cambio de signo
Relajación, raíz cúbica, $\omega = 0,5$	33	No
Relajación, lineal, $\alpha = 10^{-5}$	23	No

Tabla 3: Comparación de iteraciones entre ambas partes.

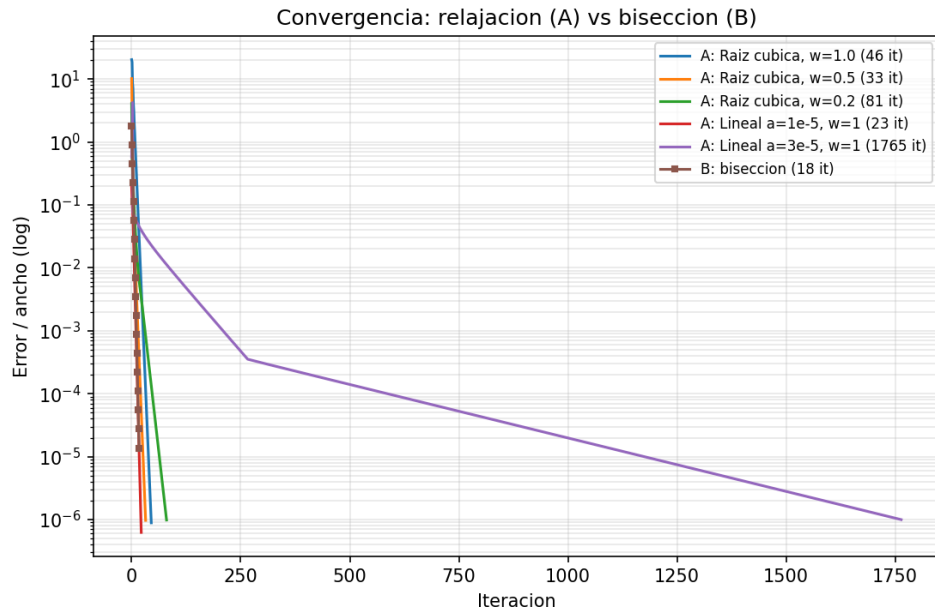


Figura 3: Convergencia de la relajación (Parte A) y de la bisección (Parte B).

4. Análisis comparativo: estabilidad de la relajación frente a la elección de $g(x, y)$

Cualquier reorganización $\mathbf{p} = \mathbf{g}(\mathbf{p})$ tiene como puntos fijos los mismos equilibrios $f_1 = f_2 = 0$; lo que cambia con $\mathbf{g}$ es la *estabilidad de la iteración*. La iteración con factor de relajación ω converge localmente si el radio espectral del jacobiano relajado,

$$\rho[(1 - \omega) I + \omega J_g] < 1, \quad J_g = \begin{pmatrix} \partial g_1 / \partial x & \partial g_1 / \partial y \\ \partial g_2 / \partial x & \partial g_2 / \partial y \end{pmatrix}, \quad (16)$$

y mientras menor es ρ , más rápida es la convergencia. La Tabla 4 resume las cinco iteraciones probadas desde (1,0, 0,5) y la Figura 4 las compara sobre el mapa de energía y por su error en cada iteración.

Formulación g	Parámetros	ρ	Iter.	Punto final	Tipo
Raíz cúbica (6)	$\omega = 1,0$	0.687	46	(5,9465, 6,2531)	mínimo
Raíz cúbica (6)	$\omega = 0,5$	0.672	33	(5,9465, 6,2531)	mínimo
Raíz cúbica (6)	$\omega = 0,2$	0.869	81	(5,9465, 6,2531)	mínimo
Lineal (7)	$\alpha = 10^{-5}$	0.55	23	(1,3333, 0)	silla
Lineal (7)	$\alpha = 3 \times 10^{-5}$	1.73	1765	(7,65, 0)	silla

Tabla 4: Radio espectral ρ (calculado en el punto al que converge, salvo la última fila, donde se calcula en (1,3333,0)) e iteraciones necesarias con $\epsilon = 10^{-6}$.

1. **La elección de g decide a qué equilibrio se llega.** Desde el mismo punto inicial, la raíz cúbica termina en el mínimo estable (5,9465, 6,2531) y la formulación lineal con $\alpha = 10^{-5}$ en la silla del eje (1,3333, 0) (Fig. 4, izquierda; ver también Fig. 1).
2. **Convergencia de la iteración no implica estabilidad física.** El punto (1,3333,0) es una silla (Fig. 2): la iteración converge allí, pero una carga real se escaparía por y por la repulsión de Coulomb.

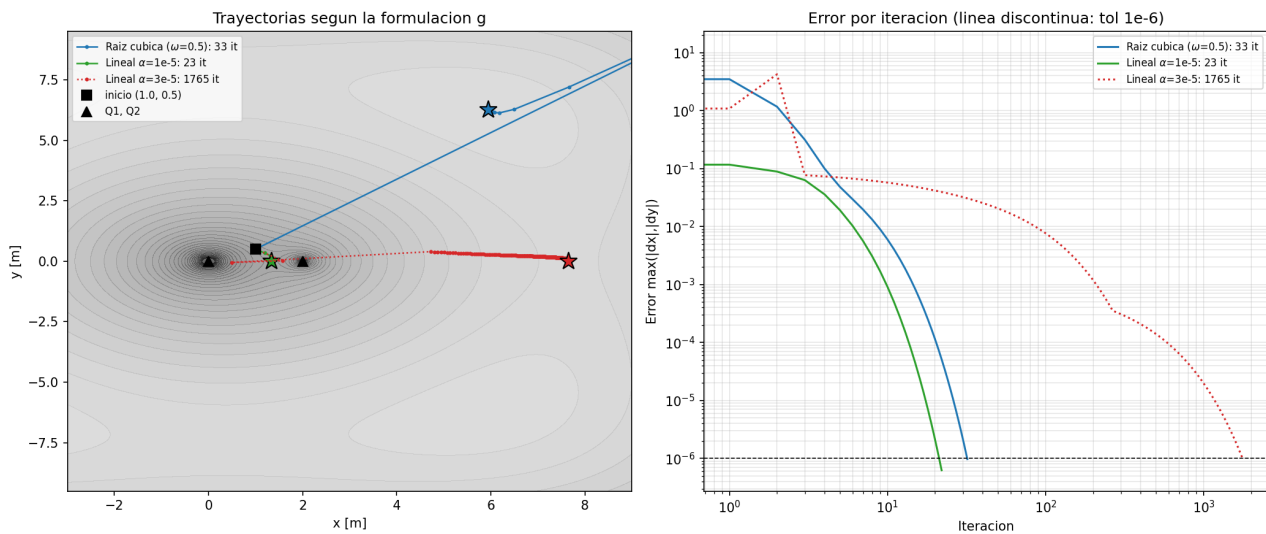


Figura 4: Izquierda: trayectorias desde $(1,0,0.5)$ sobre el mapa de energía para tres formulaciones (las estrellas marcan el punto final). Derecha: error $\max(|\Delta x|, |\Delta y|)$ por iteración.

3. **Cerca de $y = 0$ la raíz cúbica es inestable.** $g_2 \propto y^{1/3}$ tiene derivada $\frac{1}{3}y^{-2/3} \rightarrow \infty$ cuando $y \rightarrow 0$, así que el eje es repulsor de esa iteración y la trayectoria azul se aleja de él. En la formulación lineal las derivadas son finitas y basta elegir α para que $|1 + \alpha \partial f_1 / \partial x| < 1$ y $|1 - \alpha \partial f_2 / \partial y| < 1$.
4. **Hay un α máximo.** En $(1,3333, 0)$, $\partial f_1 / \partial x \approx -9,10 \times 10^4$ y $\partial f_2 / \partial y \approx 4,55 \times 10^4$, de modo que se requiere $\alpha < 2/9,10 \times 10^4 \approx 2,2 \times 10^{-5}$. Con $\alpha = 10^{-5}$ el factor mayor es 0,55 y converge en 23 iteraciones; con $\alpha = 3 \times 10^{-5}$ el factor en x es $-1,73$ ($|\cdot| > 1$), el punto $(1,3333, 0)$ deja de atraer y la iteración termina, muy lentamente (1765 iteraciones, curva roja), en otro equilibrio del eje.
5. **El factor de relajación ω tiene un valor óptimo.** Con la raíz cúbica, $\omega = 0,5$ reduce ρ de 0,687 a 0,672 y baja las iteraciones de 46 a 33; $\omega = 0,2$ amortigua demasiado ($\rho = 0,869$) y sube a 81. Sub-relajar no siempre acelera.
6. **Comparación con la bisección (Parte B).** La bisección necesitó 18 iteraciones con convergencia garantizada si hay cambio de signo, mientras que la relajación necesita verificar $\rho < 1$ para cada g y puede terminar en un equilibrio distinto del buscado.

5. Código fuente

El código completo (Partes A y B, gráficas y clasificación de equilibrios) se incluye a continuación.

```

1 # -*- coding: utf-8 -*-
2 """
3 Created on Sun Sep 27 23:07:26 2026
4
5 @author: lezly
6 """
7
8 """
9 Actividad FC2 - Equilibrio y puntos prometidos en un sistema de cargas
10 electricas no lineal.
11
12 Sistema:
13     Q1 = +4 uC en (0,0)      Q2 = +1 uC en (2,0)      q = +1 uC (movil)
14     Potencial externo no lineal Vext(x,y) = A (x^4 + y^4), A = 0.5 V/m^4

```

```

15
16 Fuerza neta (dividida por q, asi q no interviene en el equilibrio):
17     f1(x,y) = k [ Q1 x / r1^3 + Q2 (x-2) / r2^3 ] - 4 A x^3
18     f2(x,y) = k [ Q1 y / r1^3 + Q2 y / r2^3 ] - 4 A y^3
19     con r1 = sqrt(x^2+y^2), r2 = sqrt((x-2)^2+y^2)
20
21 Parte A: relajacion (punto fijo) en 2 variables, desde (1.0, 0.5), tol 1e-6.
22 Parte B: biseccion en el eje x sobre U(x) - U0 = 0, intervalo [0.1, 1.9],
23         precision 1e-5.
24 """
25
26 import numpy as np
27 import matplotlib.pyplot as plt
28 from matplotlib.colors import LogNorm
29
30 # -----
31 # 0. Constantes fisicas
32 # -----
33 k = 8.99e9 # constante de Coulomb [N m^2 / C^2]
34 Q1, Q2 = 4e-6, 1e-6
35 q = 1e-6
36 A = 0.5 # [V/m^4]
37
38
39 def radios(x, y):
40     return np.hypot(x, y), np.hypot(x - 2.0, y)
41
42
43 def f_sistema(x, y):
44     """Fuerza neta por unidad de carga: (f1, f2). Equilibrio: f1 = f2 = 0."""
45     r1, r2 = radios(x, y)
46     f1 = k * (Q1 * x / r1**3 + Q2 * (x - 2.0) / r2**3) - 4 * A * x**3
47     f2 = k * (Q1 * y / r1**3 + Q2 * y / r2**3) - 4 * A * y**3
48     return f1, f2
49
50
51 def U_total(x, y):
52     """Energia potencial total de q: U = q [kQ1/r1 + kQ2/r2 + A(x^4+y^4)]."""
53     r1, r2 = radios(x, y)
54     return q * (k * Q1 / r1 + k * Q2 / r2 + A * (x**4 + y**4))
55
56
57 # -----
58 # PARTE A - Relajacion multivariable
59 # -----
60 # Formulacion 1 (raiz cubica): como el termino no lineal es 4A x^3 (un cubo
61 # puro), se despeja directamente: x = cbrt( (k/4A) [ ... ] )
62 def g_raiz_cubica(x, y):
63     r1, r2 = radios(x, y)
64     c = k / (4 * A)
65     g1 = np.cbrt(c * (Q1 * x / r1**3 + Q2 * (x - 2.0) / r2**3))
66     g2 = np.cbrt(c * (Q1 * y / r1**3 + Q2 * y / r2**3))
67     return g1, g2
68
69
70 # Formulacion 2 (lineal): x = x + a f1, y = y - a f2.
71 # Es una reorganizacion valida (en el punto fijo f1 = f2 = 0). El signo de
72 # cada componente se elige para que |dg/dx| < 1 cerca del equilibrio.
73 def crear_g_lineal(alfa):
74     def g(x, y):
75         f1, f2 = f_sistema(x, y)
76         return x + alfa * f1, y - alfa * f2

```

```

77     return g
78
79
80 def relajacion_2var(g, p0, omega=1.0, tol=1e-6, max_iter=5000):
81     """
82     Relajacion en 2 variables:
83         x_(n+1) = (1-w) x_n + w g1(x_n, y_n)
84         y_(n+1) = (1-w) y_n + w g2(x_n, y_n)
85     Criterio de paro: max(|dx|, |dy|) < tol.
86     Devuelve trayectoria (N,2), errores y bandera de convergencia.
87     """
88     x, y = p0
89     tray = [(x, y)]
90     errores = []
91     for _ in range(max_iter):
92         gx, gy = g(x, y)
93         xn = (1 - omega) * x + omega * gx
94         yn = (1 - omega) * y + omega * gy
95         err = max(abs(xn - x), abs(yn - y))
96         errores.append(err)
97         x, y = xn, yn
98         tray.append((x, y))
99         if not np.isfinite(err) or abs(x) > 1e6 or abs(y) > 1e6:
100             return np.array(tray), errores, False
101         if err < tol:
102             return np.array(tray), errores, True
103     return np.array(tray), errores, False
104
105
106 def clasificar(x, y, h=1e-4):
107     """Clasifica el equilibrio con los valores propios del hessiano de U."""
108     def Ug(a, b):
109         return U_total(a, b)
110     Uxx = (Ug(x + h, y) - 2 * Ug(x, y) + Ug(x - h, y)) / h**2
111     Uyy = (Ug(x, y + h) - 2 * Ug(x, y) + Ug(x, y - h)) / h**2
112     Uxy = (Ug(x + h, y + h) - Ug(x + h, y - h)
113            - Ug(x - h, y + h) + Ug(x - h, y - h)) / (4 * h**2)
114     ev = np.linalg.eigvalsh(np.array([[Uxx, Uxy], [Uxy, Uyy]]))
115     if ev.min() > 0:
116         return "minimo (estable)"
117     if ev.max() < 0:
118         return "maximo"
119     return "punto silla (inestable)"
120
121
122 p0 = (1.0, 0.5)
123 tol_A = 1e-6
124
125 print("=" * 66)
126 print("PARTE A - Relajacion en 2 variables, inicio (1.0, 0.5), tol 1e-6")
127 print("=" * 66)
128
129 resultados_A = {}
130 for nombre, g, omega in [
131     ("Raiz cubica, w=1.0", g_raiz_cubica, 1.0),
132     ("Raiz cubica, w=0.5", g_raiz_cubica, 0.5),
133     ("Raiz cubica, w=0.2", g_raiz_cubica, 0.2),
134     ("Lineal a=1e-5, w=1", crear_g_lineal(1e-5), 1.0),
135     ("Lineal a=3e-5, w=1", crear_g_lineal(3e-5), 1.0),
136 ]:
137     tray, err, ok = relajacion_2var(g, p0, omega, tol_A)
138     resultados_A[nombre] = (tray, err, ok)

```

```

139     if ok:
140         xf, yf = tray[-1]
141         print(f"{nombre:22s} converge en {len(err):4d} iter -> "
142               f"({xf:.6f}, {yf:.6f}) [{clasificar(xf, abs(yf))}]")
143     else:
144         print(f"{nombre:22s} NO converge (diverge / max_iter)")
145
146 # -----
147 # PARTE B - Biseccion sobre el eje x
148 # -----
149 def U_eje(x):
150     return U_total(x, 0.0)
151
152
153 def biseccion(g, a, b, tol=1e-5, max_iter=200):
154     """Biseccion. Exige g(a)*g(b) < 0; si no, no hay raiz garantizada."""
155     if g(a) * g(b) > 0:
156         raise ValueError("g(a) y g(b) tienen el mismo signo: no hay "
157                           "cambio de signo en el intervalo")
158     anchos = []
159     for it in range(1, max_iter + 1):
160         c = 0.5 * (a + b)
161         anchos.append(b - a)
162         if g(a) * g(c) < 0:
163             b = c
164         else:
165             a = c
166         if (b - a) < tol:
167             break
168     return 0.5 * (a + b), anchos, it
169
170
171 print()
172 print("=" * 66)
173 print("PARTE B - Biseccion sobre el eje x, intervalo [0.1, 1.9], tol 1e-5")
174 print("=" * 66)
175
176 xs = np.linspace(0.1, 1.9, 20001)
177 Umin = U_eje(xs).min()
178 print(f"U(x) en [0.1, 1.9]: minimo = {Umin*1e3:.2f} mJ, "
179       f"U(0.1) = {U_eje(0.1)*1e3:.2f} mJ, U(1.9) = {U_eje(1.9)*1e3:.2f} mJ")
180
181 # Caso 1: el U0 del enunciado
182 U0 = 15e-3
183 try:
184     biseccion(lambda x: U_eje(x) - U0, 0.1, 1.9)
185 except ValueError as e:
186     print(f"\nU0 = 15 mJ -> {e}")
187     print(f" U(x) >= {Umin*1e3:.2f} mJ en todo el intervalo: el objetivo de "
188           f"15 mJ NO es alcanzable.")
189
190 # Caso 2: U0 alcanzable (entre U(1.9)=108.8 mJ y U(0.1)=364.3 mJ)
191 U0 = 150e-3
192 x_bis, anchos_B, n_B = biseccion(lambda x: U_eje(x) - U0, 0.1, 1.9)
193 print(f"\nU0 = 150 mJ -> x* = {x_bis:.6f} m en {n_B} iteraciones "
194       f"f(cota teorica: {int(np.ceil(np.log2(1.8/1e-5))))}")
195 print(f" Verificacion: U(x*) = {U_eje(x_bis)*1e3:.4f} mJ")
196
197 # -----
198 # Comparacion de convergencia A vs B
199 # -----
200 print("\nComparacion: iteraciones necesarias")

```

```

201 print(f" Biseccion (B, tol 1e-5) : {n_B}")
202 for nombre, (tray, err, ok) in resultados_A.items():
203     if ok:
204         print(f" Relajacion (A) {nombre:20s}: {len(err)}")
205
206 # -----
207 # GRAFICAS
208 # -----
209 equilibrios = [(4/3, 0.0), (0.3802, 7.3939), (5.9465, 6.2531)]
210 from scipy.optimize import fsolve
211 eq_ref = []
212 for guess in [(1.3, 0.0), (0.4, 7.4), (5.9, 6.2)]:
213     s = fsolve(lambda p: f_sistema(*p), guess)
214     eq_ref.append((s[0], s[1]))
215
216 # --- Figura 1: trayectorias de convergencia en el plano (x,y)
217 fig, ax = plt.subplots(1, 2, figsize=(13, 5.5))
218 for a_, nombre in zip(ax, ["Raiz cubica, w=0.5", "Lineal a=1e-5, w=1"]):
219     tray, err, ok = resultados_A[nombre]
220     a_.plot(tray[:, 0], tray[:, 1], 'o-', ms=3, lw=1, label="trayectoria")
221     a_.plot(*tray[0], 'gs', ms=9, label="inicio (1.0, 0.5)")
222     a_.plot(*tray[-1], 'r*', ms=14, label=f"final ({tray[-1,0]:.4f}, {tray[-1,1]:.4f})")
223     a_.plot([0, 2], [0, 0], 'k^', ms=9, label="Q1, Q2")
224     a_.set_title(f"{nombre}: {len(err)} iteraciones")
225     a_.set_xlabel("x [m]"); a_.set_ylabel("y [m]")
226     a_.grid(alpha=0.3); a_.legend(fontsize=8)
227 plt.tight_layout()
228 plt.savefig("trayectoria_relajacion.png", dpi=150)
229
230 # --- Figura 2: mapa de potencial con equilibrios superpuestos
231 X, Y = np.meshgrid(np.linspace(-3, 9, 700), np.linspace(-9.5, 9.5, 700))
232 Z = U_total(X, Y) * 1e3 # mJ
233 Z = np.clip(Z, 1.0, 300.0)
234 fig, ax = plt.subplots(figsize=(8, 7))
235 cs = ax.contourf(X, Y, Z, levels=np.logspace(np.log10(1.0), np.log10(300.0), 60),
236                 norm=LogNorm(), cmap="viridis")
237 cb = fig.colorbar(cs, label="U(x,y) [mJ] (escala log, recortado a 1-300 mJ)")
238 cb.set_ticks([1, 3, 10, 30, 100, 300]); cb.set_ticklabels(["1", "3", "10", "30", "100", "300"])
239 ax.plot([0, 2], [0, 0], 'w^', ms=10, label="Q1, Q2")
240 for (ex, ey) in eq_ref:
241     for sgn in [1, -1] if abs(ey) > 1e-6 else [1]:
242         tipo = clasificar(ex, sgn * ey)
243         marc = 'r*' if "minimo" in tipo else 'ro'
244         ax.plot(ex, sgn * ey, marc, ms=11, mec='w')
245 ax.plot([], [], 'r*', ms=11, mec='w', label="equilibrio: minimo")
246 ax.plot([], [], 'ro', ms=8, mec='w', label="equilibrio: silla")
247 ax.set_xlabel("x [m]"); ax.set_ylabel("y [m]")
248 ax.set_title("Mapa de energia potencial y puntos de equilibrio")
249 ax.legend(loc="upper right", fontsize=8)
250 plt.tight_layout()
251 plt.savefig("mapa_potencial.png", dpi=150)
252
253 # --- Figura 3: convergencia A vs B
254 plt.figure(figsize=(7.5, 5))
255 for nombre, (tray, err, ok) in resultados_A.items():
256     if ok:
257         plt.plot(err, label=f"A: {nombre} ({len(err)} it)")
258 plt.plot anchos_B, 's-', ms=3, label=f"B: biseccion ({n_B} it)")
259 plt.yscale('log'); plt.xlabel("Iteracion"); plt.ylabel("Error / ancho (log)")
260 plt.title("Convergencia: relajacion (A) vs biseccion (B)")
261 plt.grid(alpha=0.3, which='both'); plt.legend(fontsize=8)
262 plt.tight_layout()

```

```
263 plt.savefig("convergencia_A_vs_B.png", dpi=150)
264
265 # Mostrar las tres figuras en pantalla (sin esto solo se guardan como PNG)
266 plt.show()
267
268 print("\nEquilibrios (fsolve) y su clasificacion:")
269 for (ex, ey) in eq_ref:
270     print(f"  ({ex:.5f}, {ey:.5f}) -> {clasificar(ex, abs(ey) if ey else 0.0)}")
```

Código 1: Solución de la Actividad FC2.